Week 6

Topic: Sliding Window & Prune and Search

---

79. Word Search (Medium)

Given an m x n grid of characters board and a string word, return true *if* word *exists in the grid*.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCCED"

Output: true

For detailed descriptions and constraints, please refer to Leetcode 79.

154. Find Minimum in Rotated Sorted Array II(Hard)

Suppose an array of length n sorted in ascending order is **rotated** between 1 and n times. For example, the array nums = [0,1,4,4,5,6,7] might become:

- [4,5,6,7,0,1,4] if it was rotated 4 times.
- [0,1,4,4,5,6,7] if it was rotated 7 times.

Notice that **rotating** an array [a[0], a[1], a[2], ..., a[n-1]] 1 time results in the array [a[n-1], a[0], a[1], a[2], ..., a[n-2]].

Given the sorted rotated array nums that may contain **duplicates**, return *the minimum element of this array*.

You must decrease the overall operation steps as much as possible.

Example 1:

Input: nums = [1,3,5]

Output: 1

Example 2:

Input: nums = [2,2,2,0,1]

Output: 0

For detailed descriptions and constraints, please refer to Leetcode 154.

710. Random Pick with Blacklist (Hard)

You are given an integer n and an array of **unique** integers blacklist. Design an algorithm to pick a random integer in the range [0, n - 1] that is **not** in blacklist. Any integer that is in the mentioned range and not in blacklist should be **equally likely** to be returned.

Optimize your algorithm such that it minimizes the number of calls to the **built-in** random function of your language.

Implement the Solution class:

- Solution(int n, int[] blacklist) Initializes the object with the integer n and the blacklisted integers blacklist.
- int pick() Returns a random integer in the range [0, n - 1] and not in blacklist.

E xample 1:

Input

["Solution", "pick", "pick", "pick", "pick", "pick", "pick", "pick"]

[[7, [2, 3, 5]], [], [], [], [], [], [], []]

Output

[null, 0, 4, 1, 6, 1, 0, 4]

Explanation

Solution solution = new Solution(7, [2, 3, 5]);

solution.pick(); // return 0, any integer from [0,1,4,6] should be ok. Note that for every call of pick,

// 0, 1, 4, and 6 must be equally likely to be returned (i.e., with probability 1/4).

solution.pick(); // return 4

solution.pick(); // return 1

solution.pick(); // return 6

solution.pick(); // return 1

solution.pick(); // return 0

solution.pick(); // return 4

For detailed descriptions and constraints, please refer to Leetcode 710.